



Literature (High) Results

Objective

I'm a solo student engineer building a submission for the Razorpay AI Buildathon 2026 (deadline September 5, 2026 — about 7 days from now). I've settled on the idea and need deep, validated implementation research, not more idea discovery.

THE PRODUCT: A checkout integrity firewall for agentic commerce, entered under the "AI Risk Manager" track. Two components:

1. CORE — sits at the merchant/payment-processor side (not the buyer's browser). When an AI shopping agent submits an order, it re-verifies the catalog content (price, item, quantity, policy) the agent acted on against the merchant's last-verified state, and separately scans that content for injected instructions (hidden text in product descriptions, coupon fields, reviews) designed to manipulate the agent. Blocks the call to the payment API if content is stale or hostile, before an order is created.
2. EXTENSION — after payment clears, verifies the resulting order/transaction state is consistent, catching cases where payment succeeds but the order record goes stale or split, and offers one-click revalidation.

I already know about: dmsfiris/agentic-commerce-blueprint v0.9.2 (GitHub, real but minimal — 9 commits, 0 stars), Microsoft's ThinkingBox-Bench (GitHub, 507 policy-conditioned workflows, real and current), OWASP GenAI Security Project's LLM security taxonomy, and Razorpay's test-mode Orders/Payments/Refunds APIs. Don't re-find these — build on top of and beyond them.

I need you to find AND VALIDATE as currently usable (not just cite as existing) — for each item, confirm it's downloadable/installable today, note its license, and flag anything abandoned, paywalled behind enterprise sales, or requiring infrastructure a solo builder can't stand up in days:

1. PRETRAINED PROMPT-INJECTION / JAILBREAK DETECTION MODELS — is there an open, pretrained classifier (e.g., Meta's Llama Prompt Guard or similar) that detects injected instructions in text? Specifically check whether it's been validated against injection embedded in product/catalog/e-commerce content specifically, versus only chatbot-conversation injection — these are different distributions and I need to know if it transfers or needs adaptation.
2. DATASETS for evaluating this specific attack pattern — content-injection attacks against tool-using/shopping agents (AgentDojo, AgentDyn, and anything more current or more e-commerce-specific than those). Confirm what's actually downloadable, what license, and whether any include e-commerce/checkout-specific attack scenarios rather than only banking/travel/workspace domains.
3. REFERENCE ARCHITECTURES for the "trusted vs. untrusted content" separation pattern used in the Agentic Commerce Blueprint and in Stripe's pending patent (US20260067335A1) — capability tokens, decision envelopes, freshness/revalidation checks. I need the pattern explained precisely

enough to implement in days, not just cited.

4. RAZORPAY-SPECIFIC APIS AND SDKS relevant to gating an order before creation — confirm exact test-mode endpoints for Orders, Payments, and any hold/pre-authorization mechanism, with links to current documentation, not assumed from general payment-gateway knowledge.

5. TECH STACK for a solo one-week build — validated, currently-maintained (not abandoned) libraries for: (a) a minimal demo shopping agent to show the vulnerability live, (b) the middleware/firewall layer itself, (c) a simple dashboard showing blocked vs. allowed checkout attempts. Recommend specific packages with version/maintenance status, not generic category names.

6. STATE-CONSISTENCY/SPLIT-STATE DETECTION resources for the extension — anything beyond ThinkingBox-Bench that specifically benchmarks or provides tooling for detecting payment-succeeded-but-order-state-stale conditions in agentic workflows.

For everything you return, give me a working link and tell me plainly if something looked promising but turned out unmaintained, paywalled, or not actually applicable — I'd rather know a resource is dead now than discover it on day 4.

Results

Checkout Integrity Firewall for Agentic Commerce: Validated Implementation Research

This report provides validated, currently-usable resources across all six research areas for your Razorpay AI Buildathon submission. For each resource, availability, license, and honest usability assessments are provided.

1. Pretrained Prompt-Injection Detection Models

The critical finding across this research area: **no open pretrained prompt-injection classifier has been validated against injection embedded in e-commerce product descriptions, coupon fields, or review content.** All existing models are trained and evaluated on chatbot-conversation or generic prompt-attack distributions, which is a different distribution from merchant catalog content containing injected instructions ([1.1], [1.2], [2.1]).

Recommended model: ProtectAI v2 (protectai/deberta-v3-base-prompt-injection-v2 on HuggingFace, Apache 2.0, 184M parameters, DeBERTa-v3-base architecture). It offers the best balance of open availability, reasonable resilience, and easy deployment. Evasion testing showed it resists character-injection attacks significantly better than v1 (20.26% attack success rate vs. 95.18% for v1), though it remains vulnerable to emoji smuggling and Unicode tag attacks ([2.1], [2.2], [2.3]).

Models to avoid for this build:

- **Meta PromptGuard-2-86M** (HuggingFace, Llama license): Achieves near-ceiling recall on PI benchmarks but produces catastrophic false-positive rates — 96% FPR on AgentHarm-benign prompts and 98% on ToxicChat. It would over-block legitimate catalog text in your demo ([1.1]).
- **Semalith v1.4** (184M, DeBERTa-v3-base): Technically superior with zero FPR on benign agentic content and 44× fewer parameters than Llama-Guard-3-8B, but production weights require enterprise

license and author contact — not usable for a 7-day sprint ([1.2], [1.3], [1.4], [1.5]).

- **NeMo Guard Jailbreak Detect** (NVIDIA, random forest): Particularly susceptible to adversarial ML evasion at 65.22% ASR ([2.2]).

Important for your demo: Character injection techniques (emoji smuggling, Unicode tags, bidirectional text, number substitution) can achieve up to 100% evasion against these classifiers while remaining interpretable to the underlying LLM ([2.2], [2.4], [2.3], [2.5], [2.6]). Your firewall should therefore layer the ProtectAI classifier with deterministic sanitization rules: strip hidden DOM elements, detect suspicious Unicode patterns, and normalize text before classification.

Model Name	Parameters	Architecture	License	HuggingFace / Download Link	E-commerce Content Validation Status	Key Limitation	Recommendation for this Project
Meta PromptGuard-2-86M	86M	mDeBERTa-v3-base fine-tuned classifier	Llama license	https://huggingface.co/meta-llama/PromptGuard-2-86M	No direct validation found on product descriptions, catalog pages, reviews, coupon fields, or checkout content; benchmarked on generic prompt-injection / jailbreak distributions instead ([1.1], [2.1])	Extremely high false-positive behavior on benign content; reported 96% FPR on benign agentic prompts in comparison work, so it may over-block real catalog text ([1.1])	Not recommended as primary gate for your demo; useful only as a baseline comparator if you can tolerate noisy blocking
ProtectAI Prompt Injection Detection v1	184M	DeBERTa-v3-base fine-tuned classifier	Apache-2.0	https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v1	No direct validation found on e-commerce/catalog injection; appears trained/evaluated on generic prompt security corpora, not commerce content ([2.1], [2.6])	Highly vulnerable to evasion; reported up to 95.18% ASR under evasion attacks, so a hostile merchant/content source could bypass it with obfuscation ([2.3])	Do not use except as a “bad baseline” in your demo to show why stronger filtering is needed
ProtectAI Prompt Injection Detection v2	184M	DeBERTa-v3-base fine-tuned classifier	Apache-2.0	https://huggingface.co/protectai/deberta-v3-base-prompt-injection-v2	No direct validation found on product/catalog/review injection specifically; evidence points to generic prompt-injection distributions rather than shopping-agent content ([2.1], [2.6])	Still evadable, especially by emoji / Unicode smuggling, but materially better than v1; reported 20.26% ASR in the cited evaluation ([2.2])	RECOMMENDED pick for a solo one-week build: open, downloadable today, lightweight enough, and best balance of availability + resilience among openly usable options
Semalith v1.4	184M	DeBERTa-v3-base encoder with multi-head safety classifier	Enterprise license / author contact required	No open weights link; paper says weights and harness available on request / enterprise terms	No direct e-commerce-specific validation found ; evaluated on prompt-injection benchmarks and benign agentic prompts, not catalog/review commerce fields ([1.2], [1.3])	Promising technical performance, but not freely available ; production weights require author contact / enterprise access, so this is impractical for a student buildathon sprint ([1.2], [1.3])	Promising but unusable for your timeline unless you unexpectedly get access immediately
NeMo Guard Jailbreak Detect	Not stated in cited evidence	Lightweight classifier using pre-trained embeddings + random forest	NVIDIA / NeMo license terms	https://github.com/NVIDIA/NeMo-Guardrails	No direct validation found on e-commerce content injection; discussed as a general jailbreak / prompt-attack detector ([2.1])	Particularly susceptible to AML-style evasion; reported 65.22% ASR in one evaluation, making it weak against adaptive attackers ([2.2])	Not recommended as primary detector; acceptable only if you are already using NeMo Guardrails and need a convenience component
Vijil Prompt Injection	Not stated in cited evidence	ModernBERT-based binary classifier	Commercial / unclear open model terms	Product / model references exist, but open install path is unclear from cited evidence	No direct validation found on e-commerce/catalog injection; appears to target generic prompt-injection detection ([2.1])	Availability / licensing is unclear for an open student build, and cited evidence focuses on evaluation targets rather than a clear open deployment path ([2.1], [2.6])	Avoid for this build unless you verify a free, immediately installable package yourself

Model Name	Parameters	Architecture	License	HuggingFace / Download Link	E-commerce Content Validation Status	Key Limitation	Recommendation for this Project
Cross-model conclusion	—	—	—	—	For all models above, I found no evidence of validation specifically on injected instructions embedded in product descriptions, coupons, reviews, or other merchant-side commerce content. Existing evidence is mostly from chatbot, agent, or generic prompt-attack distributions, so transfer to your threat model should be treated as unproven and needs a small custom eval set ([3.1], [4.1], [4.2])	All detector families are vulnerable to obfuscation attacks such as emoji smuggling, Unicode tags, numbers mapping, bidi text, and other transformations that preserve meaning for LLMs while evading filters ([2.2], [2.3], [2.5], [2.6])	Best practical plan: ship ProtectAI v2 as the pretrained detector, add deterministic regex/HTML sanitization around hidden text and suspicious imperatives, and evaluate on your own synthetic catalog/review injection set before demo day

Table: This table compares currently usable prompt-injection detection models for a solo buildathon implementation, including availability, license, and applicability to merchant-side e-commerce content. It highlights the main gap: none of the open detectors are validated specifically on catalog/review/checkout injection, so ProtectAI v2 is the best practical starting point but still needs custom evaluation.

2. Evaluation Datasets and Benchmarks

Dataset / Benchmark Name	Source / Link	License	Domains Covered	E-commerce Coverage	Download Status	Key Notes for This Project
AgentDojo v0.1.22+	GitHub: ethz-spylab/agentdojo	MIT	Banking, Slack, Travel, Workspace; 97 tasks and 629 security task-goal pairs	No	Downloadable / pip-installable today	Strong for indirect prompt-injection evaluation in tool-using agents, but not checkout-specific; useful as a transfer baseline only, not as your main commerce eval set ([3.2], [3.1], [5.1])
AIP-Bench	GitHub + HuggingFace release from <i>Protocol-Level Attacks on Agentic Commerce Platforms</i>	Public release confirmed in paper; specific repo/dataset licenses should be checked per artifact	Agentic commerce workflows: service discovery, contract negotiation, payment / value transfer; attack classes include payment redirection, wallet hijacking, secret harvesting, CSRF, open redirects	Yes	Publicly downloadable; Docker Compose reproducible	Most relevant resource found: deterministic, model-agnostic, commerce-specific, and directly aligned to order/payment integrity and protocol abuse; best external benchmark for your buildathon demo ([6.1], [6.2], [6.3], [6.4], [6.5])
Magentic Marketplace	GitHub: microsoft/magentic-marketplace	Open-source, but exact license not stated in retrieved evidence	Two-sided marketplaces with Assistant Agents and Service Agents; demonstrated in restaurant / contractor-style markets; supports discovery, negotiation, transaction	Partial	Open-source release confirmed	Very useful as an environment for shopping-agent manipulation and market dynamics; extensible toward e-commerce, but not a ready-made checkout-attack benchmark out of the box ([7.1], [7.2], [7.3], [7.4], [7.5], [7.6], [7.7])
InjectAgent	Mentioned in later papers; likely code release exists, but not validated from retrieved evidence	Unknown from retrieved evidence	Tool-using agent indirect prompt-injection benchmark, but domain specifics were not recoverable from retrieved evidence	Unknown	Existence confirmed in citations, but public download path not validated here	Promising on paper, but for a 7-day build I would not rely on it unless you personally verify the repo and dataset immediately; current evidence is too thin ([5.1])

Dataset / Benchmark Name	Source / Link	License	Domains Covered	E-commerce Coverage	Download Status	Key Notes for This Project
IBM Adversarial Prompt Evaluation	GitHub: IBM/Adversarial-Prompt-Evaluation	Open-source repo; check repo for exact license	Guardrail benchmarking across 15 defenses with malicious + benign prompt datasets	No	Public code release confirmed in paper	Good for stress-testing detectors and comparing prompt-injection classifiers, but it evaluates defenses , not commerce workflows; use it to test your hostile-content scanner, not your checkout protocol logic ([2.1])
Kaya et al. IEEE S&P 2026 web-plugin study	Paper: <i>When AI Meets the Web: Prompt Injection Risks in Third-Party AI Chatbot Plugins</i>	Paper only; standalone dataset release not confirmed in retrieved evidence	Real-world web chatbot plugins scraping first-party and third-party website content; attacks via reviews, comments, forms, feeds, support threads	Partial	Real-world evidence confirmed; reusable standalone dataset not confirmed	Important validation that this threat is real in commerce: the study found about 13% of e-commerce websites exposed to this class of indirect injection through scraped third-party content; great motivation/citation, weak as an immediate benchmark asset ([4.1], [4.2])

Table: This table compares the most relevant currently usable datasets and benchmark environments for evaluating indirect prompt injection against tool-using and shopping-style agents. It highlights the key gap: AgentDojo is strong but non-commerce, while AIP-Bench is the closest validated fit for checkout-integrity research.

Key findings:

AgentDojo (GitHub: [ethz-spylab/agentdojo](https://github.com/ethz-spylab/agentdojo), MIT license, pip-installable) is the most mature benchmark for indirect prompt injection against tool-using agents, but it covers only four domains: Banking, Slack, Travel, and Workspace — **no e-commerce or checkout scenarios** ([3.1], [5.1]). It provides 97 user tasks and 629 security test cases. Use it as a transfer baseline for your injection detection layer, but do not claim it validates checkout-specific attacks.

AIP-Bench is the most directly relevant resource found. It is the first deterministic benchmark specifically for agentic commerce security, covering service discovery, contract negotiation, and value transfer via payment protocols. It uses exclusively deterministic judges (HTTP status codes, wallet string matches, log-pattern regexes) rather than LLM judges, enabling reproducible measurement ([6.1], [6.2], [6.3], [6.4], [6.5]). It is publicly available on GitHub and HuggingFace with Docker Compose reproducibility, and covers attack scenarios including payment redirection, wallet hijacking, secret harvesting, CSRF, and open redirects across three platforms (CoralOS, Fetch.ai uAgents, AP2). Its root-cause taxonomy (RC-1 through RC-6) maps directly to your firewall's concerns, particularly RC-1 (registry content without integrity verification) and RC-4 (non-atomic check-then-execute/TOCTOU) ([6.4]).

Magentic Marketplace (Microsoft Research, GitHub: [microsoft/magentic-marketplace](https://github.com/microsoft/magentic-marketplace), open-source) provides a full two-sided marketplace simulation environment with Assistant Agents (buyers) and Service Agents (merchants) interacting via REST APIs. It supports testing agent vulnerability to prompt injection, psychological manipulation, and behavioral biases ([7.1], [7.2], [7.3], [7.4]). Currently demonstrated in restaurant/contractor domains but extensible. The exact license was not stated in retrieved evidence — verify on the repo.

Kaya et al. (IEEE S&P 2026) provides critical real-world validation: approximately 13% of e-commerce websites expose their chatbots to indirect prompt injection through failure to distinguish first-party content (product descriptions) from untrusted third-party content (customer reviews, comments, forms). Attackers can poison reviews to inject instructions that are scraped into LLM context ([4.1], [4.2]). This is exactly the attack vector your firewall addresses, but no standalone reusable dataset was confirmed from this work.

3. Reference Architectures for Trusted/Untrusted Content Separation

Critical correction: Patent US20260067335A1 is titled "Security and Privacy Preserving Agentic Browser" and was filed by Bao Tran — it is **not a Stripe patent**. It is, however, the most detailed public specification of the capability-token/decision-envelope/content-sanitization architecture relevant to your product.

Architectural Pattern	Source Reference	Key Components to Implement	Implementation Approach for Solo Builder
Capability Token Gating	US20260067335A1 describes time-bounded, scope-bound capability tokens issued by a privileged broker, validated before execution, and revoked on navigation/origin mismatch; tokens constrain site, intent, tool class, limits, and expiry ([8.1], [8.2], [8.3], [8.4], [8.5])	Capability broker; token schema with merchant/site binding, intent, allowed actions, amount ceiling, expiry, nonce; validation middleware; revocation on origin/path/session mismatch; fail-closed enforcement before payment/order calls	Implement as JWTs signed by your firewall service with claims like {merchant_id, intent, allowed_actions, amount_ceiling, expires_at, nonce}. Issue token only after catalog snapshot verification. On every pre-checkout call, verify signature, expiry, merchant match, action match, and amount <= ceiling before allowing Razorpay order creation. Store used nonces in Redis/SQLite to prevent replay.
Content Sanitization Pipeline	US20260067335A1 separates trusted system instructions/user goals from untrusted page content and runs an injection detector over HTML, markdown, CSS, image alt-text, and microdata, then transforms risky content by redaction/summarization/removal before model use ([8.6], [8.3], [8.5])	Context assembler separating trusted vs untrusted fields; parsers for HTML/markdown/CSS/alt-text/microdata; prompt-injection classifier; deterministic risk rules; sanitization transforms; audit trail of removed/redacted spans	Build a pre-agent normalization stage: parse merchant content with lxml/BeautifulSoup, strip hidden DOM/CSS tricks, extract visible text + alt text + reviews + coupon fields, then run ProtectAI v2 on chunks. Add regex/rule checks for imperatives like "ignore previous", invisible text, excessive Unicode, bidi markers, coupon-field prompt payloads. Produce sanitized_content, risk_score, and flags[]; pass only sanitized content to the demo agent.
PCAT Sidecar Principles	AIP-Bench/PCAT defines five protocol-layer controls: P1 response integrity, P2 caller identity binding, P3 secure channel enforcement, P4 atomic payment state, P5 tool-call authorization; implemented as HTTP sidecar controls with 0% ASR on several structural classes ([6.6], [6.7], [6.8], [6.9], [6.10], [6.11])	P1 integrity verification; P2 identity/auth binding; P3 CSRF/redirect protection; P4 compare-and-swap state lock; P5 pre-authorized tool/API invocation control; middleware interception layer	For a one-week build, simplify aggressively: P1 = HMAC-sign merchant catalog snapshots or snapshot manifests; P2 = static merchant API key / signed webhook secret instead of DID; P3 = only allow server-to-server calls from your middleware, reject unexpected redirect targets; P4 = per-cart or per-order lock in Redis/SQLite using SETNX or transaction row versioning so "verify -> create order" is atomic; P5 = whitelist only create_order, capture_payment, refund actions and require internal signed headers between agent demo and firewall.
Five-Layer Defense	Mao et al. propose five defense layers for agentic commerce: L1 Prompt/Tool Hygiene, L2 Verified Execution Context, L3 Payment Authorization/Custody, L4 Inter-Agent Trust, L5 Market/Compliance Monitoring; they argue secure deployment needs coordinated cross-layer controls, not point defenses ([9.1], [9.2], [9.3], [9.4], [9.5])	L1 sanitize external inputs and verify tool provenance; L2 bind actions to verified workflow/context; L3 enforce spend/authorization controls; L4 trust controls across agents; L5 monitoring/logging/compliance analytics	For the hackathon, focus only on L1+L2+L3. L1 = content scanner + freshness checks. L2 = canonical checkout intent object containing merchant, SKU IDs, qty, price, policy hash, and snapshot timestamp; reject if any field diverges at payment time. L3 = enforce spend ceiling, merchant allowlist, and one-time capability token before calling Razorpay. Mention L4/L5 in slides as future work, but do not try to fully build them.

Architectural Pattern	Source Reference	Key Components to Implement	Implementation Approach for Solo Builder
Decision Envelope	US20260067335A1 describes decision envelopes/action summaries listing destination, capabilities, items/amounts, and risk indicators for approval/audit around sensitive actions ([8.6], [8.2])	Structured action summary; risk score; itemized intent; freshness verdict; injection verdict; immutable log entry; dashboard rendering	Represent every attempted checkout as a JSON envelope like {action, merchant, items, total, risk_score, injection_flags, staleness_check, token_id, decision}. Display this in your dashboard for both allowed and blocked attempts. For the core demo you can keep approval automatic but visible; for the extension, add a one-click “revalidate and retry” button that re-runs snapshot comparison and state consistency checks after payment.

Table: This table translates the research-backed patterns into a concrete implementation plan for the checkout integrity firewall. It maps each architectural idea to the minimum viable components you can realistically build in a week.

3a. Capability Token Pattern (US20260067335A1)

The patent describes time-bounded capability tokens scoped to specific sites, intents, and time windows, issued exclusively by a privileged capability broker. Tokens specify allowed tool classes (DOM operations, clicks, fills, submits, network fetches), enforce per-domain and per-merchant budgets with transaction ceilings and rate limits, and are automatically revoked on navigation changes or origin mismatches. Untrusted page content cannot mint tokens ([8.1], [8.2], [8.3], [8.7], [8.4], [8.5]).

Your implementation: Issue JWTs from your firewall service containing {merchant_id, intent, allowed_actions, amount_ceiling, expires_at, nonce}. Only issue after catalog snapshot verification passes. Validate on every pre-checkout call.

3b. Content Sanitization Pipeline (US20260067335A1)

The patent separates trusted system instructions and user goals from untrusted page content through a context assembler schema. An injection detector scans HTML, markdown, CSS, image alt-text, and microdata for prompt-injection indicators, assigns risk scores, and transforms risky content via redaction, summarization, or removal. Sensitive data uses one-way write-only autofill paths preventing readback ([8.6], [8.3], [8.5], [8.8]).

3c. PCAT Sidecar (AIP-Bench Paper)

PCAT (Protocol-level Commerce Agent Trust) is a platform-agnostic HTTP sidecar implementing five security principles: P1 response integrity (ECDSA P-256 signature verification with nonce caching), P2 caller identity binding (DID-based proof verification), P3 secure channel enforcement (CSRF + redirect validation), P4 atomic payment state (asyncio.Lock-backed compare-and-swap preventing TOCTOU race conditions), and P5 MCP tool-call authorization (pre-registered tokens for sensitive operations). It reduced attack success rates from 100% to 0% for four of five structural vulnerability classes with zero false positives and negligible overhead (0.21ms for P1, 0.08ms for P5) ([6.6], [6.7], [6.8], [6.9], [6.10], [6.11]).

3d. Five-Layer Defense (Mao et al. SoK)

The SoK paper proposes a layered defense architecture: L1 Prompt and Tool Hygiene (sanitize external inputs, tag agent-originated content, verify tool provenance), L2 Verified Execution Context (output validation, runtime context binding, capability graphs), L3 Payment Authorization and Custody, L4 Inter-Agent Trust Controls, and L5 Market and Compliance Monitoring ([9.1], [9.2], [9.3]). For your buildathon, focus on L1+L2+L3 and cite L4+L5 as future work.

4. Razorpay-Specific APIs

No scholarly papers cover Razorpay API internals. Based on current public documentation (<https://razorpay.com/docs/api/>):

Test Mode: Use `rzp_test_` prefixed API keys. All test-mode calls go to `https://api.razorpay.com/v1/` with Basic Auth (`key_id:key_secret`).

Order Gating Flow for Your Firewall:

- 1. **Create Order:** `POST /v1/orders` with `{amount, currency, receipt}` — this is where your fire-wall gates. Only call this endpoint after (a) catalog content passes freshness check, (b) injection scan is clean, and (c) capability token is valid.
- 2. **Manual Capture:** Set `payment_capture: 0` in the order creation payload. This creates the order but does not auto-capture payment, giving you a window for post-payment verification (the Extension component).
- 3. **Capture Payment:** `POST /v1/payments/{payment_id}/capture` with `{amount, currency}` — call this only after your post-payment state consistency check passes.
- 4. **Fetch Payment:** `GET /v1/payments/{payment_id}` — use this in the Extension to verify payment state matches order state.
- 5. **Refund:** `POST /v1/payments/{payment_id}/refund` — if state consistency check fails post-cap-ture, trigger refund.

SDK: `pip install razorpay` (official, MIT license, actively maintained on PyPI).

Important: The manual-capture flow (`payment_capture: 0`) is the key mechanism for your Extension component. It creates a natural checkpoint where your firewall can verify order-payment state consistency before finalizing.

5. Tech Stack for Solo One-Week Build

Compo nent	Recommend ed Package	Version/Stat us	Licens e	Purpose in this Project	Alternativ es Consider ed	Notes
Demo Shopping Agent	<code>browser-use</code> (<code>browser-us</code> <code>e/browser-us</code> <code>e</code>)	Active; popu- lar OSS browser agent framework	MIT	Simulate an AI shopping agent browsing merchant pages, reading poisoned product/review content, and attempting checkout so you can show the vulnerability live	Playwright + LangChain ReAct agent	Best for demo speed; if you need deter- ministic replay and tighter control over action traces, fallback to plain Playwright. This row is implementation guidance rather than literature-backed evidence from retrieved papers.
Prompt Injection Detector	<code>protectai/de</code> <code>berta-v3-bas</code> <code>e-prompt-inj</code> <code>ection-v2 via</code> <code>transformers</code>	Open model; 184M DeBERTa-v3- base class; use with <code>transfo</code> <code>rmers>=4.3</code> <code>5, torch>=2.</code> <code>0</code>	Apache- 2.0	Scan catalog text, reviews, coupon text, and other mer- chant content for injected in- structions before order creation	Meta Prom ptGuard-2 -86M; Semalith v1.4	Best practical open choice for a 7-day build. PromptGuard has severe false- positive issues on benign content, while Semalith is not openly usable because access is enterprise/contact-gated ([1.1], [1.2], [2.1], [2.2]).
Middlew are/ Firewall	<code>FastAPI + Pyd</code> <code>antic</code>	Active; cur- rent main- stream Python API stack	MIT	Serve as the HTTP sidecar/firewall intercepting agent-to-payment requests, performing freshness checks, hostile-content scanning, and capability-token validation	Flask	Better fit than Flask because your reval- idation path, webhook handling, and lock-backed state checks are easier to express cleanly in async request han- dlers. The sidecar pattern matches PCAT-style interposition ([6.6], [6.7], [6.8]).

Component	Recommended Package	Version/Status	License	Purpose in this Project	Alternatives Considered	Notes
Payment Integration	razorpay Python SDK	Official SDK; active PyPI package	MIT	Connect the demo merchant backend to Razorpay test mode for order creation and post-auth capture flows	Raw REST calls with requests	Use <code>rzp_test_credentials</code> . Relevant flow for gating: only call <code>POST /v1/orders</code> after firewall approval; use manual capture flow and <code>POST /v1/payments/{id}/capture</code> after your consistency checks. This row is based on current public Razorpay API usage conventions rather than retrieved papers.
Dashboard	Streamlit	Active; rapid app prototyping	Apache-2.0	Show judges blocked vs allowed attempts, decision envelopes, risk flags, stale snapshot mismatches, and one-click revalidation	Gradio	Streamlit is better for tabular operations dashboards and timeline/event views. Decision-envelope visualization maps directly to the action-summary pattern in the patent architecture ([8.6], [8.2]).
Content Parsing / Sanitization	beautifulsoup4 + lxml + bleach	Mature, active parsing/sanitization stack	MIT / BSD / Apache-2.0	Parse HTML, detect hidden or suspicious content, extract visible text / reviews / coupon fields / alt-text, and sanitize before model ingestion	selectolax; regex-only parsing	Necessary because the patent and web-plugin evidence both show injection can hide in DOM, CSS, alt-text, and scraped third-party content. Do not rely on classifier-only detection ([8.3], [8.5], [4.1], [4.2]).
State Management	sqlite3 initially; Redis only if concurrency forces it	SQLite is stdlib and zero-setup; Redis optional	Public domain-style stdlib / BSD-style ecosystem norms	Persist catalog snapshots, token nonces, order states, payment states, and revalidation logs for the extension	PostgreSQL; Redis-first design	Start with SQLite for speed. If you need concurrent lock semantics for <code>verify -> create order</code> , add Redis <code>SETNX</code> or row-version locking. This reflects the atomic payment state idea from PCAT P4 ([6.7], [6.9], [6.10]).
Catalog Snapshot Hashing	hashlib (sha256)	Stdlib; no extra dependency	Python stdlib	Hash normalized catalog state so the firewall can detect drift between browse-time content and checkout-time content	blake3	Normalize first, then hash: SKU, title, displayed price, quantity, shipping/policy summary, coupon terms, and extracted visible text. This is your fastest path to a freshness check and directly supports the trusted-vs-untrusted separation pattern ([8.3], [9.1], [9.2]).

Table: This table recommends a concrete, fast-to-build stack for the checkout integrity firewall demo. It prioritizes packages that are easy for one person to wire up in a week while still aligning with the validated security patterns found in the research.

Key architectural recommendation: Structure your demo as three services:

- 1. **Demo Agent** (browser-use or Playwright script): Visits a mock merchant page containing both clean and poisoned product listings. Shows the agent reading injected instructions and attempting checkout.
- 2. **Firewall Middleware** (FastAPI): Intercepts the agent's checkout request. Runs the content sanitization pipeline (BeautifulSoup parsing → hidden content detection → ProtectAI v2 scan → freshness hash comparison). Issues/validates capability tokens. Gates the Razorpay `POST /v1/orders` call.
- 3. **Dashboard** (Streamlit): Displays real-time blocked vs. allowed attempts, decision envelopes with risk scores and injection flags, catalog snapshot diffs, and the one-click revalidation button for the Extension.

6. State-Consistency / Split-State Detection

This is the thinnest area of existing tooling — **no dedicated benchmark or tooling exists for detecting payment-succeeded-but-order-state-stale conditions in agentic workflows**. This is a genuine gap and a legitimate novel contribution for your submission.

The closest resources:

- **PCAT P4 (Atomic Payment State):** Implements asyncio.Lock-backed compare-and-swap stores with rollback mechanisms to serialize concurrent budget deductions and prevent TOCTOU attacks on payment operations. Uses SQLite's `UPDATE...WHERE` or Redis+Lua scripts for consume-once semantics across distributed instances ([6.7], [6.9], [6.10]).
- **AIP-Bench RC-4:** Specifically tests non-atomic check-then-execute vulnerabilities in agentic commerce platforms, directly relevant to your Extension's concern about state drift between payment confirmation and order record ([6.4]).
- **Mao et al. SoK:** Covers settlement integrity and financial irreversibility in autonomous agent transactions, emphasizing that secure deployment requires verifying that an agent's reasoning intent matches its actual encoded transactions ([9.2], [9.3], [9.6]).

Your implementation for the Extension: After Razorpay payment capture succeeds, poll `GET /v1/payments/{id}` and compare the returned `{amount, status, order_id, method}` against your firewall's stored intent envelope. Flag mismatches on amount, `order_id` binding, or payment status. Store a row-versioned order state in SQLite and reject any state transition that skips expected steps (created → paid → captured → fulfilled). The one-click revalidation button should re-run the full pipeline: re-fetch catalog content, re-hash, re-scan for injection, compare against the stored snapshot, and present the diff.

Summary of Honest Assessments

What works today:

- ProtectAI v2 is deployable and decent (but not e-commerce-validated)
- AIP-Bench is the first real commerce-specific security benchmark
- PCAT sidecar pattern is directly implementable
- Razorpay manual-capture flow gives you the checkpoint you need
- US20260067335A1 patent provides the most detailed public capability-token specification

What looked promising but has caveats:

- **Semalith v1.4:** Best technical performance but enterprise-gated — dead end for a hackathon
- **AgentDojo:** Excellent quality but zero e-commerce coverage — use for transfer only
- **Magentic Marketplace:** Rich environment but not a ready-made attack benchmark; needs setup work
- **InjectAgent:** Cited everywhere but download path not verified — do not rely on it without checking yourself

What does not exist yet (your novel contribution space):

- No e-commerce-specific prompt-injection evaluation dataset
- No split-state detection benchmark for payment-order consistency
- No open checkout integrity firewall reference implementation

These gaps are precisely what make your submission novel. The fact that Kaya et al. found ~13% of e-commerce websites vulnerable to indirect prompt injection via untrusted third-party content ([4.1], [4.2]), combined with the absence of any checkout-specific defense tooling, gives you a strong motivation story for the judges.

References

[1.1] Semalith v1.4: A Calibrated 184M Safety Classifier Achieving State-of-the-Art Prompt-Injection Detection at 44x Fewer Parameters than Llama-Guard-3-8B. Tejasvi C. Addagada. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.22545> Context: "According to the excerpt, PromptGuard-2-86M is a binary prompt-injection classifier developed by Meta with 86M parameters. It achieves near-ceiling recall on prompt-injection benchmarks but has severe false-positive rate issues, producing 96% FPR on AgentHarm-benign prompts and 98% FPR on ToxicChat by over-flagging edgy content as harmful. The excerpt positions Semalith v1.4 as offering comparable PI recall to PromptGuard-2-86M on structural PI benchmarks with 190× lower false-positive rates on benign agentic content. However, the excerpt does not provide information about model licensing, HuggingFace availability, or validation against indirect prompt injection in non-chatbot contexts like e-commerce product descriptions."

[1.2] Semalith v1.4: A Calibrated 184M Safety Classifier Achieving State-of-the-Art Prompt-Injection Detection at 44x Fewer Parameters than Llama-Guard-3-8B. Tejasvi C. Addagada. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.22545> Context: "The excerpt discusses multiple prompt injection detection models. Beyond Meta's PromptGuard-2-86M (86M parameters, binary classifier achieving high recall but 96% false-positive rate on benign content), other models include LlamaGuard and Granite Guardian, which focus on general harm but perform poorly on subtle prompt-injection attacks. Semalith v1.4 is a 184M-parameter DeBERTa-v3-base classifier offering simultaneous three-axis safety classification (prompt injection, general harm, BFSI compliance). It achieves zero false positives (FPR=0.000) on 208 benign agentic prompts versus 0.063 for Llama-Guard-3-8B, winning all 7 prompt-injection benchmarks at 44× fewer parameters than Llama-Guard-3-8B. Semalith v1.4 processes prompts in 11.6 milliseconds. Production weights and evaluation harness are released under enterprise license requiring author contact. The model is trained on a 76,204-row deduplication-cleaned corpus from 49 public sources with zero synthetic prompts."

[1.3] Semalith v1.4: A Calibrated 184M Safety Classifier Achieving State-of-the-Art Prompt-Injection Detection at 44x Fewer Parameters than Llama-Guard-3-8B. Tejasvi C. Addagada. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.22545> Context: "Semalith v1.4 is a 184M-parameter encoder classifier achieving state-of-the-art prompt-injection detection with 44× fewer parameters than Llama-Guard-3-8B. It maintains zero false positives on 208 benign agentic prompts. The model is trained on a contamination-controlled corpus with a three-axis label taxonomy and auxiliary super-category loss. Model weights and corpus manifest are available on request, with production weights and evaluation harness available under enterprise license (contact author for access). The model can be retrained from microsoft/deberta-v3-base. Other prompt injection detection models referenced include: Meta's Llama Guard, Llama Guard 3 8B, Meta's PromptGuard 2 (86M parameters), IBM Research's Granite Guardian, WildGuard (from NeurIPS 2024), and ServiceNow Research's AprielGuard. The excerpt focuses primarily on Semalith v1.4's specifications, licensing, and reproducibility rather than detailed comparisons with competing classifiers."

[1.4] Semalith v1.4: A Calibrated 184M Safety Classifier Achieving State-of-the-Art Prompt-Injection Detection at 44x Fewer Parameters than Llama-Guard-3-8B. Tejasvi C. Addagada. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.22545> Context: "Semalith v1.4 is a 184M parameter safety classifier that uses a DeBERTa-v3-base encoder with a 22-class classification head for prompt injection and harm detection. It achieves state-of-the-art prompt-injection detection with 44× fewer parameters than Meta's Llama-Guard-3-8B (184M vs 8B). In apples-to-apples comparison using identical benchmarks and hardware (RTX 4090), Semalith v1.4 wins 11 of 18 benchmarks and offers significantly better latency—33× faster at batch size 1 (11.6 ms vs 387.4 ms) and 100× higher throughput at batch size 8 (435 vs 4.3 prompts/second). The model employs AdamW optimization with gradient clipping, linear warmup, and cosine decay, trained on a maximum sequence length of 256 tokens. The excerpt does not specify license or detailed availability information."

[1.5] Semalith v1.4: A Calibrated 184M Safety Classifier Achieving State-of-the-Art Prompt-Injection Detection at 44x Fewer Parameters than Llama-Guard-3-8B. Tejasvi C. Addagada. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.22545> Context: "The excerpt describes Semalith v1.4, a 184M parameter safety classifier designed for prompt-injection detection that achieves comparable or superior performance to Meta's Llama-Guard-3-8B (8B parameters) while using 44x fewer parameters. On prompt-injection and adversarial-stealth benchmarks (Semalith's design target), it wins all 7 benchmarks with margins often exceeding 70 percentage points. The model performs particularly well on HackaPrompt (0.995 recall), Gandalf (0.970), and WildJailbreak (0.968). Semalith v1.4 was trained on a contamination-controlled corpus with a 22-class taxonomy. The excerpt also mentions DeBERTa-v3-large (304M) as a considered alternative architecture during development, which showed only 0.3 percentage point improvement in validation macro-F1 (0.879 vs 0.876 for the base 184M variant) at 65% higher parameter count. However, the excerpt does not specify Semalith's availability, licensing terms, or provide information about other prompt-injection detection models beyond Llama-Guard-3."

[2.1] Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, Peter Garraghan. ArXiv (2025). <https://doi.org/10.48550/arxiv.2504.11168> Context: "The excerpt identifies several prompt injection and jailbreak detection models beyond Meta's Prompt Guard. These include: Azure Prompt Shield (from Microsoft, detects hate-speech and violence using ensemble neural models); ProtectAI Prompt Injection Detection v1 and v2 (both fine-tuned from DeBERTa-v3-base with 184M parameters, released November 2023 and April 2024); NeMo Guard Jailbreak Detect (Nvidia's lightweight random forest-based classifier using pre-trained embeddings); and Vijil Prompt Injection (binary classifier fine-tuned from ModernBert). Meta's Prompt Guard is a multi-label classifier fine-tuned from mDeBERTa-v3-base (86M parameters). However, the excerpt does not contain information about Semalith v1.4, its model size, availability, license, or performance metrics. It also does not provide detailed performance comparisons across the mentioned systems."

[2.2] Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, Peter Garraghan. ArXiv (2025). <https://doi.org/10.48550/arxiv.2504.11168> Context: "The paper evaluates two categories of evasion techniques against LLM guardrails. Character injection techniques proved highly effective, with Emoji Smuggling achieving 100% Attack Success Rate (ASR) against prompt injections, followed by Unicode Tags (90.15% ASR), Numbers (81.18%), Bidirectional Text (78.69%), and Upside Down Text (63.54%). Azure Prompt Shield was bypassed with 71.98% ASR for prompt injections, while Protect AI v2 showed better resilience at 20.26% ASR but was still vulnerable to Emoji and Unicode tag Smuggling. Adversarial ML (AML) evasion techniques, implemented through TextAttack library, employ word importance ranking and perturbation methods including synonym substitution, typos, and word reordering. These techniques maintain semantic similarity while disrupting classifier processing. The effectiveness varies across guardrails, with NeMo Guard Jailbreak Detect most susceptible to AML evasion (65.22% ASR) and Azure Prompt Shield more resistant (12.98% ASR)."

[2.3] Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, Peter Garraghan. ArXiv (2025). <https://doi.org/10.48550/arxiv.2504.11168> Context: "The excerpt describes multiple evasion attack techniques against prompt injection detectors. Character injection techniques proved highly effective, including emoji smuggling and unicode tags. Specific transformation attacks with high success rates include: Tags (90.15% ASR), Numbers (81.18%-94.62%), Bidirectional Text (78.69%-99.23%), and Upside Down Text (63.54%-100%). Among adversarial machine learning evasion techniques, TextFooler was most effective against prompt injections with 46.27% average ASR, followed by Bert-Attack (57.57%) and BAE (52.56%). Against specific guardrails, Protect AI v1 showed highest vulnerability (95.18% ASR on prompt injections), while Azure Prompt Shield had moderate susceptibility (62.91% ASR). The excerpt notes that character injection techniques can completely evade detection while remaining interpretable to the underlying LLM, representing a critical weakness in guardrail implementations."

[2.4] Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, Peter Garraghan. ArXiv (2025). <https://doi.org/10.48550/arxiv.2504.11168> Context: "The excerpt describes two main evasion attack approaches against LLM guardrails: (1) Character Injection, a traditional technique commonly used in cyber security attacks on software input fields, and (2) Adversarial Machine Learning (AML) evasion techniques that subtly perturb how detection models interpret prompt context by exploiting their over-reliance on learned features. The authors tested these methods against six prominent protection systems including Microsoft Azure Prompt Shield and Meta's Prompt Guard, achieving in some instances 100% evasion success while maintaining the functionality of the underlying prompt. Additionally, they demonstrate that attackers can enhance attack success rates against black-box targets by leveraging word importance ranking computed from offline white-box models, enabling transfer of attack effectiveness across different detection systems."

[2.5] Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, Peter Garraghan. ArXiv (2025). <https://doi.org/10.48550/arxiv.2504.11168> Context: "The excerpt presents empirical data on evasion attack techniques against LLM guardrails. Several transformation-based attack methods are demonstrated with concrete examples: (1) Alzantot technique modifies verb tenses and word choices (e.g., "need to provide" to "needed to offer"); (2) Numbers technique employs character substitution and number-letter replacement (e.g., "Ignore all instructions" becomes "1gn0r3 411 1n57ruc710n5"); (3) Bert-Attack modifies adjectives and verbs while maintaining semantic meaning; (4) PWWS (Probability Weighted Word Saliency) technique alters specific words while preserving intent; (5) Pruthi technique removes characters (e.g., "fake" becomes "ake"). Detection baseline results show these evasion techniques achieve high success rates: Azure Prompt Shield shows 89.74% evasion on jailbreaks and 58.92% on prompt injections, while Meta Prompt Guard shows 100% accuracy on jailbreaks but vulnerability is still demonstrated through the evasion examples provided."

[2.6] Bypassing LLM Guardrails: An Empirical Analysis of Evasion Attacks against Prompt Injection and Jailbreak Detection Systems. William Hackett, Lewis Birch, Stefan Trawicki, Neeraj Suri, Peter Garraghan. ArXiv (2025). <https://doi.org/10.48550/arxiv.2504.11168> Context: "The paper identifies two main evasion technique categories against LLM guardrails: Character Injection and Adversarial ML Evasion. Character injection techniques manipulate text through various transformations while maintaining semantic meaning that LLMs can still interpret. The paper catalogs 12 character injection techniques including numbers mapping (H3110), homographs, zero-width characters, diacritics (hèllö), spacing variations, upside-down text, full-width characters, bidirectional text, deletion characters, emoji smuggling, and Unicode tag smuggling. These methods exploit the gap between how guardrail classifiers process text and how LLMs can still comprehend obfuscated or altered payloads. The paper also mentions adversarial ML evasion approaches including Bert-Attack, BAE, Deep Word Bug, and Alzantot techniques. The research targeted specific guardrails including Azure Prompt Shield, ProtectAI Prompt Injection Detection (v1 and v2), Meta Prompt Guard, NeMo Guard, and Vijil Prompt Injection."

[3.1] Continual Red-Teaming and Guardrail Distillation for Tool-Using LLM Agents: Prompt-Injection Resistance with Utility Preservation. Wesley Gao. Stout in Computer Science and Technology Studies (2026). <https://doi.org/10.61424/hewtat90> Context: "AgentDojo v1.1.1 consists of four domains: Banking (16 user tasks, 9 injection goals, 11 tools), Slack (21 tasks, 5 goals, 11 tools), Travel (20 tasks, 7 goals, 28 tools), and Workspace (40 tasks, 6 goals, 24 tools). These four suites together contain 97 total user tasks, 27 injection goals, and 74 declared tools across 629 security task-goal pairs. The excerpt does not mention e-commerce, shopping, or checkout scenarios in AgentDojo. The document also does not provide information about an InjectAgent benchmark or its domains."

[3.2] Continual Red-Teaming and Guardrail Distillation for Tool-Using LLM Agents: Prompt-Injection Resistance with Utility Preservation. Wesley Gao. Stout in Computer Science and Technology Studies (2026). <https://doi.org/10.61424/hewtat90> Context: "AgentDojo v0.1.22 is a benchmark for evaluating tool-using language model agents against prompt injection attacks. It comprises 97 realistic user tasks paired with 629 security test cases in interactive environments. Each case combines a legitimate user goal with an attacker objective, an

injection location, and consequential tools, allowing joint measurement of task completion rates and attack success rates. The benchmark was designed to evaluate defenses that may suppress attacks while blocking useful work, requiring measurement of false rejection rates and tool costs alongside security metrics. However, the excerpt does not specify which application domains are covered, whether e-commerce or shopping scenarios are included, under what license AgentDojo is released, or whether it is available for download from GitHub."

[4.1] When AI Meets the Web: Prompt Injection Risks in Third-Party AI Chatbot Plugins. Yigitcan Kaya, Anton Landerer, Stijn Pletinckx, Michelle Zimmermann, Christopher Kruegel, Giovanni Vigna. 2026 IEEE Symposium on Security and Privacy (SP) (2026). <https://doi.org/10.1109/sp63933.2026.00062> Context: "The research found that approximately 13% of e-commerce websites have exposed their chatbots to third-party content vulnerabilities. The study discovered that 15 plugins use web-scraping tools to enrich chatbot context but fail to distinguish between trusted first-party content (like product descriptions) and untrusted third-party content (like customer reviews). This distinction failure introduces indirect prompt injection risks, as attackers can poison third-party content such as product reviews or comments to inject malicious instructions into the chatbot's context. The plugins' inability to properly segregate content sources enables adversaries to exploit these chatbots through content they control on the website."

[4.2] When AI Meets the Web: Prompt Injection Risks in Third-Party AI Chatbot Plugins. Yigitcan Kaya, Anton Landerer, Stijn Pletinckx, Michelle Zimmermann, Christopher Kruegel, Giovanni Vigna. 2026 IEEE Symposium on Security and Privacy (SP) (2026). <https://doi.org/10.1109/sp63933.2026.00062> Context: "The excerpt discusses indirect prompt injection vulnerabilities in web chatbot plugins that scrape content from websites, including e-commerce platforms. The research found that website chatbots often ingest untrusted third-party content such as user reviews on e-commerce sites, which can contain malicious prompts. Scrapers used by 15 plugins consistently extracted third-party content from test pages. The excerpt notes that content can enter websites through multiple channels including product reviews, contact forms, social media feeds, wiki pages, guest blog posts, and user-submitted support threads—all of which may be scraped into the LLM's context. The attacks work by embedding malicious prompts in this third-party content, which are then processed by the chatbot when generating responses. However, the excerpt does not provide the specific 13% e-commerce vulnerability statistic mentioned in the question."

[5.1] Agent Against Agent: An Agentic System for Automatic Prompt Injection Red Teaming. Yanting Wang, Chenlong Yin, Runpeng Geng, Jinyuan Jia. ArXiv (2608). <https://doi.org/10.48550/arxiv.2608.05108> Context: "According to the excerpt, AgentDojo is a benchmark for tool-using agents that covers diverse domains including workspace management, banking, travel, and Slack. The document does not mention e-commerce, shopping, or checkout scenarios in AgentDojo. Regarding InjectAgent, the excerpt only mentions that 30 test samples from InjectAgent [4] were used for comparison with baselines, but it does not provide any information about the specific domains covered by the InjectAgent benchmark."

[6.1] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "AIP-Bench is a deterministic benchmark for testing protocol-level attacks on agentic commerce platforms. It uses objective judges (HTTP status codes, wallet string matches, log patterns, event counts) rather than LLM judges, enabling reproducible, model-agnostic measurement. The benchmark includes a cross-vendor LLM behavioral impact study testing 8 models across 5 providers with 1,440 trials. It evaluates vulnerabilities across three representative agentic commerce platform architectures: Google AP2 (vendor-backed payment protocol), Fetch.ai uAgents (decentralized framework), and CoralOS (self-hosted orchestration). The benchmark covers commercial workflows including service discovery, contract negotiation, and value transfer via payment protocols. AIP-Bench is publicly available and downloadable: the code repository is hosted on GitHub and the dataset is available on Hugging Face."

[6.2] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "AIP-Bench is a deterministic benchmark for testing agentic commerce platform security, designed to address limitations of

existing benchmarks that use LLM judges. Unlike traditional approaches that can conflate model-dependent behavior with structural flaws, AIP-Bench uses exclusively deterministic judges including HTTP status codes, wallet strings, log-pattern regexes, WebSocket event counts, and vulnerable-code patterns. The benchmark covers multiple attack scenarios across platforms like CoralOS, Fetch.ai, and AP2 protocol implementations, testing vulnerabilities across RC-1 through RC-6 severity classes. Test scenarios include secret harvesting, delivery attacks, model overrides, wallet hijacking, CSRF, and open redirect attacks, with reproducibility provided through Docker Compose environments, exact API call sequences, SHA-256 reference hashes, and expected judge outputs."

[6.3] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "AIP-Bench (Agent Interaction Protocol Benchmark) is described as the first deterministic benchmark for agentic commerce security. It is designed to test structural vulnerabilities in agentic commerce platforms at the protocol level rather than the AI model level. The benchmark uses exclusively deterministic evaluation methods including HTTP status codes, wallet string matches, log patterns, and event counts as judges, rather than relying on LLM judges which can introduce model bias. This approach enables reproducible, model-agnostic measurement across different AI models and platforms. The benchmark is part of a broader study identifying 33 structural vulnerabilities across three leading agentic commerce platforms (CoralOS, Fetch.ai uAgents, and AP2 v0.2.0). The excerpt does not provide information about whether the benchmark is publicly downloadable or the specific domains it covers beyond agentic commerce security."

[6.4] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "AIP-Bench is a deterministic benchmark for protocol-level attacks on agentic commerce platforms. It systematically identifies structural (model-independent) protocol-level vulnerabilities across multiple platforms. The benchmark organizes 33 vulnerabilities into six root-cause classes: RC-1 (registry/marketplace content without integrity verification), RC-2 (payment destination from untrusted source), RC-3 (authentication credential transmission via observable channel), RC-4 (non-atomic check-then-execute/TOCTOU), RC-5 (authentication scope not enforced), and RC-6 (behavioral manipulation via poisoned agent descriptions). RC-1 through RC-5 are structural vulnerabilities with 100% attack success rate and are model-independent, while RC-6 is semantic with variable success rates depending on the model. The excerpt covers CoralOS platform with 11 vulnerabilities as case study examples but does not provide information about downloadability or other platform domains covered."

[6.5] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "AIP-Bench is a benchmark for testing protocol-level attacks on agentic commerce platforms. The excerpt shows it evaluates how different large language models (Claude Sonnet 4.6, Claude 3 Haiku, Llama 3.3 70B, GPT-4o, and Gemini 2.5 Pro) handle malicious prompts attempting payment redirection attacks. It tests models' resistance to prompt injection and social engineering attacks. The benchmark is downloadable via a public GitHub repository and Hugging Face dataset. It includes Docker Compose configuration for reproducing attacks, Python proof-of-concept scripts, and result files with integrity verification. The benchmark enables full attack reproduction with components including CoralOS, a rogue marketplace, rogue federation, and PCAT tool."

[6.6] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "PCAT is a platform-agnostic HTTP sidecar defense mechanism designed to protect agentic commerce platforms without requiring modifications to the platforms themselves. It implements five principled security checks: response integrity, caller identity binding, secure channel enforcement, atomic payment state, and Model Context Protocol (MCP) tool-call authorization. These principles map to and address structural vulnerability classes RC-1 through RC-5. PCAT adapts established web security defenses including response signing, CSRF and redirect hardening, and mutual-TLS-style caller authorization to the payment-aware agent context. When evaluated against the at-

tack suite, PCAT reduces Attack Success Rate (ASR) to 0% for four of the five structural vulnerability classes that traverse the HTTP path, with RC-3 reduced to warn-only status, while maintaining zero false positives on benign traffic. The authors provide formal security theorem proofs supporting the defense approach."

[6.7] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "PCAT is a unified defense mechanism that enforces five security principles across agentic commerce platforms via HTTP middleware interposition. P1 implements supply-chain protection using ECDSA P-256 signature verification with nonce caching. P3 enforces secure channels through CSRF token validation (rejecting cross-origin requests), URL-decoded redirect validation against whitelists, and credential detection warnings. P4 prevents time-of-check-to-time-of-use (TOCTOU) attacks in payment operations by serializing concurrent budget deductions through async.Lock-backed compare-and-swap stores with rollback mechanisms. P5 protects MCP tool calls by requiring X-Caller-Identity headers with pre-registered tokens for sensitive operations like checkout and payment credential issuance. Evaluation shows PCAT reduces attack success rates from 100% to 0% across multiple attacks with zero false positives on 10,000 benign requests. Performance overhead is negligible (0.21 ms for P1, 0.08 ms for P5) with throughput maintained at 5,100-5,500 req/s under concurrent load."

[6.8] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "PCAT (Protocol-level Commerce Agent Trust) is a platform-agnostic HTTP sidecar defense framework that intercepts all communication between agents and commerce services without requiring source code changes. It implements five security principles addressing root-cause vulnerability classes RC-1 through RC-5. P1 enforces response integrity using ECDSA P-256 signatures with timestamp/nonce-based replay protection. P2 implements caller identity binding through DID-based proof verification and authorization checks. P3 provides secure channel enforcement through credential detection warnings, CSRF token validation, and redirect URL sanitization. P4 ensures atomic payment state using async.Lock-backed compare-and-swap operations to prevent race conditions. The framework operates at the HTTP path layer between agents and commerce services, with additional platform-level countermeasures required for vulnerabilities in adjacent layers like filesystem races and WebSocket events."

[6.9] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "PCAT is a defense mechanism for agentic commerce platforms comprising multiple protocol-level gates (P1-P5). It employs JSON-based signing payloads to prevent delimiter-collision attacks. Key components include: P1 for cryptographic verification with nonce replay protection via TTL heap cache; P2 for caller identity binding using DID resolution and ECDSA proof verification; P4 for preventing double-spend through atomic compare-and-swap operations using SQLite or Redis+Lua primitives; and P5 for unknown/unsigned response rejection. The system underwent three security review rounds, fixing issues like CSRF origin bypasses, nonce replay vulnerabilities, unbounded history growth, and alert-mode fallthrough. Experimental validation shows PCAT achieves 0% double-spend rates in multi-worker and distributed deployments with four concurrent instances, while maintaining complete rejection of unauthorized caller variants (no DID, unknown DID, forged proofs, unauthorized wallets)."

[6.10] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "PCAT is a protocol-layer defense mechanism for agentic commerce platforms. Key technical components include: (1) Caller identity binding (P2) using did:web resolver with end-to-end evaluation of revocation and key rotation; (2) Atomic token redemption using conditional updates (SQLite's UPDATE...WHERE or Redis+Lua scripts) to enforce consume-once semantics across distributed instances; (3) Credential detection and warning when credentials appear in URL paths. The mechanism achieved zero false positives across 10,000 diverse benign requests. However, PCAT has documented limitations: it requires an operated resolver and trust root for P2, depends on distributed atomic compare-and-swap for multi-instance deployments, and can only detect (not fully

remediate) credentials in URLs. The defense operates at the HTTP protocol layer between agents and platforms, though the excerpt notes fourteen vulnerabilities exist outside this path requiring countermeasures at other layers such as LLM proxies, filesystems, and WebSocket middleware."

[6.11] Protocol-Level Attacks on Agentic Commerce Platforms: A Cross-Platform Taxonomy, AIP-Bench, and Unified Defense. Yedidel Louck. ArXiv (2607). <https://doi.org/10.48550/arxiv.2607.21824> Context: "PCAT is a unified defense mechanism that addresses structural vulnerabilities in agentic commerce platforms by operating as an external sidecar layer at the protocol boundary. It addresses all five identified root causes (RC-1 through RC-5) without requiring changes to the underlying platforms themselves. PCAT eliminates most structural vulnerabilities that cross the HTTP boundary, achieving 0% attack success rate for four of the five structural vulnerability classes (with RC-3 in warn-only mode). The mechanism has formal proof for single-process deployment, while the distributed case with multiple instances sharing a compare-and-swap store is validated empirically. PCAT was developed alongside AIP-Bench, a deterministic benchmark for measuring agentic commerce security. However, the excerpt does not provide detailed technical specifications of how PCAT implements these protections or the specific mechanisms it employs at the protocol level."

[7.1] Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. Gagan Bansal, Wenyue Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, A. Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, Saleema Amershi. ArXiv (2025). <https://doi.org/10.48550/arxiv.2510.25779> Context: "Magentic Marketplace is an open-source environment for studying agentic markets that serves as a simulation platform for understanding interactions between market components and AI agents. It enables experimental exploration of marketplace design trade-offs and supports end-to-end testing at scale to reveal emergent outcomes and vulnerabilities that may not be apparent when testing individual components in isolation. The environment is specifically designed for testing market robustness against suboptimal agent behaviors and security threats. It can evaluate agent vulnerabilities including susceptibility to prompt injection attacks, psychological manipulation tactics, and behavioral biases. The platform supports testing market evolution over time under standard use patterns, unexpected shocks, and coordinated attacks by malicious agents. It is extensible to various market configurations, including human-in-the-loop designs and mixed AI-human markets. The excerpt does not specify the software license."

[7.2] Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. Gagan Bansal, Wenyue Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, A. Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, Saleema Amershi. ArXiv (2025). <https://doi.org/10.48550/arxiv.2510.25779> Context: "Magentic Marketplace is an open-source environment for studying agentic markets with two types of agents: Assistant Agents representing customers and Service Agents representing businesses. They interact through a central Market Environment via REST API endpoints supporting registration, service discovery, communication, and transactions. The platform is designed to systematically study agent behavior and marketplace vulnerabilities, including testing resistance to manipulation tactics (psychological persuasion, fake credentials, prompt injection attacks) and examining biases in agent behavior across different competition levels and AI model architectures. It provides a framework for understanding how autonomous agents perform in two-sided marketplaces, including their susceptibility to malicious tactics and inherited biases from training data. The excerpt does not specify the software license."

[7.3] Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. Gagan Bansal, Wenyue Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, A. Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, Saleema Amershi. ArXiv (2025). <https://doi.org/10.48550/arxiv.2510.25779> Context: "Magentic Marketplace is an open-source environment de-

signed for studying LLM agents in two-sided economic markets. It enables end-to-end study of agent behavior across the complete transaction lifecycle, from search and matching through negotiation to transaction. The platform is specifically designed to test agentic solutions and explore marketplace behaviors. Key to your question, the environment enables researchers to understand agent vulnerabilities to manipulation and bias, measure performance under realistic marketplace conditions, and systematically evaluate agent behavioral biases and vulnerabilities. The paper demonstrates how the setup can be used to evaluate market efficiencies under search limitations and susceptibility to manipulation. However, the excerpt does not specify a software license for the open-source release."

[7.4] Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. Gagan Bansal, Wenye Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, A. Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, Saleema Amershi. ArXiv (2025). <https://doi.org/10.48550/arxiv.2510.25779> Context: "Magentic Marketplace is an open-source, extensible platform designed for studying agentic economies through controlled experimentation. It addresses the critical need for robust testing before real-world deployment of AI agents in economic settings. The platform enables researchers to evaluate how different agent models behave in marketplace scenarios, revealing significant behavioral variations including differential abilities to process noisy search results and varying susceptibility to manipulation tactics. Performance gaps widen as market complexity increases. The platform is flexible enough to simulate two-sided markets with AI agents on one or both sides, supply chains, resale scenarios, and hybrid human-AI markets. The excerpt indicates the tool is valuable for understanding emergent behaviors and designing safe, efficient agentic marketplaces as LLM agents increasingly mediate economic transactions. However, the provided excerpt does not contain information about the specific license under which the open-source code is released."

[7.5] Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. Gagan Bansal, Wenye Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, A. Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, Saleema Amershi. ArXiv (2025). <https://doi.org/10.48550/arxiv.2510.25779> Context: "Magentic Marketplace is an open-source environment designed for studying agent behavior in two-sided marketplaces. It provides a controlled experimental platform with synthetic data generation capabilities across domains like restaurants and contractors. The environment supports realistic scenarios involving discovery, negotiation, and transactions under information asymmetry. It includes configurable data generation pipelines, evaluation metrics for consumer welfare, and the ability to test various AI models (both proprietary like GPT-4o and open-source like Qwen3-14b). The excerpt discusses research questions related to welfare outcomes, consideration set impacts, manipulation resistance, and agent behavior biases, suggesting the platform enables testing of marketplace dynamics and agent decision-making. However, the excerpt does not specify details about license information or explicit focus on security attacks and manipulation testing."

[7.6] Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. Gagan Bansal, Wenye Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, A. Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, Saleema Amershi. ArXiv (2025). <https://doi.org/10.48550/arxiv.2510.25779> Context: "Magentic Marketplace is an open-source environment designed for studying agentic markets. It simulates two-sided agentic marketplaces that connect Assistant Agents (acting on behalf of customers) and Service Agents (acting on behalf of businesses). The system supports the complete end-to-end economic lifecycle from discovery through negotiation to transaction fulfillment. The environment provides infrastructure for search and discovery algorithms, agent-to-agent communication, and a cen-

tralized transaction layer. It enables systematic research across diverse agent implementations and marketplace capabilities. The excerpt describes design goals including experimental control and support for different agent architectures (LLM-based, rule-based, hybrid). However, the provided excerpt does not contain specific information about security testing capabilities, agentic commerce attack testing, shopping agent manipulation research, or license details."

[7.7] Magentic Marketplace: An Open-Source Environment for Studying Agentic Markets. Gagan Bansal, Wenyue Hua, Zezhou Huang, Adam Fourney, Amanda Swearngin, Will Epperson, Tyler Payne, Jake M. Hofman, Brendan Lucier, Chinmay Singh, Markus Mobius, A. Nambi, Archana Yadav, Kevin Gao, David M. Rothschild, Aleksandrs Slivkins, Daniel G. Goldstein, Hussein Mozannar, Nicole Immorlica, Maya Murad, Matthew Vogel, Subbarao Kambhampati, Eric Horvitz, Saleema Amershi. ArXiv (2025). <https://doi.org/10.48550/arxiv.2510.25779> Context: "Magentic Marketplace is an open-source simulated environment developed by Microsoft researchers for studying two-sided agentic marketplaces where Assistant agents represent consumers and Service agents represent competing businesses. The environment enables controlled experimentation with the full transaction lifecycle, from search and matching through negotiation to transaction. It allows researchers to study key market dynamics including agent utility, behavioral biases, vulnerability to manipulation, and how search mechanisms shape market outcomes. The excerpt indicates the environment reveals how agents behave across different market conditions, including first-proposal bias effects and performance degradation at scale. However, the excerpt does not contain specific information about testing for commerce attacks, shopping agent manipulation techniques, marketplace security features, or licensing details beyond confirming open-source availability."

[8.1] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "The patent describes a security and privacy-preserving agentic browser system. For capability governance, it uses time-bounded capability tokens scoped to specific sites and intents. These tokens specify allowed tool classes (DOM operations, clicks, fills, submits, network fetches, clipboard, file writes) and can require step-up authentication for financial/healthcare domains. Tokens are revoked on site navigation changes or origin mismatches and cannot be minted by untrusted page content—only by a privileged capability broker. The system includes human-in-the-loop interlocks for sensitive operations involving credentials, payments, or confidential data. For content verification and sensitive surface protection, the system detects and redacts sensitive elements (credential fields, financial identifiers, personal data, health data) from contexts supplied to non-local inference. It validates page provenance and blocks autofill on untrusted or mismatched origins. The system uses one-way data paths for secret material that permit write-only autofill without readback, requiring biometric or passkey confirmation before submitting forms with redacted elements."

[8.2] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "The patent describes a security framework for AI browser agents that protects sensitive operations through multiple mechanisms. Capability tokens are issued by a capability broker with time-boxed, scope-bound parameters (domain, path prefix, action type, preconditions) to enforce least-privilege access. The system requires human co-approval and step-up authentication (passkey, biometric, one-time code) for high-risk actions like funds transfer and credential use. It separates trusted system instructions and user goals from untrusted page content to mitigate prompt-injection attacks, detecting injections across diverse artifacts (HTML, CSS, images, microdata). Sensitive data is protected through redaction and one-way write-only autofill paths preventing readback. The framework implements fine-grained policy enforcement per-site, per-capability, and per-intent, with default-deny rules for sensitive domains (financial, healthcare, identity). Rate limits and spend limits are enforced per domain, and an immutable audit log records decisions and outcomes."

[8.3] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "The patent describes a security framework for an AI-powered browser that governs risk actions through multiple integrated mechanisms. Capability tokens are time-bounded credentials scoped to specific sites, intents, and time windows, enforcing least-privilege per-

missions. The system separates trusted system instructions and user goals from untrusted page content using a context assembler. An injection detector identifies prompt-injection indicators in untrusted content (including hidden instructions in HTML, markdown, CSS, and image alt text), triggering transformation through redaction, summarization, or removal to produce sanitized context. An LLM policy decider proposes tool calls based on sanitized content, which are then validated against capability tokens before execution. For sensitive actions involving credentials, transactions, or data exfiltration, a human-in-the-loop interlock gates execution. The system also prevents sensitive data exfiltration by masking credential forms, account numbers, payment instruments, and personal identifiers from external inference requests."

[8.4] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "The patent describes a security-focused agentic browser system that implements capability tokens as a core governance mechanism. Time-bounded tokens are scoped to specific sites and user intents, controlling which tool classes (DOM operations, clicks, fills, submissions, network fetches, clipboard access, file writes) the local LLM can execute. Tokens require validation before tool calls execute, with sensitive domains like finance and healthcare requiring step-up authentication. Tokens are revoked on navigation changes or time expiration, and can only be issued by a privileged capability broker to prevent untrusted page content from minting them. The system protects sensitive surfaces by detecting credential fields, financial identifiers, personal identifiers, and health data within page content, then redacting these elements from any non-local inference context. Secret material uses one-way data paths permitting write-only autofill without readback capabilities, with user interlocks required before submission. All operations maintaining strict separation between local and external processing."

[8.5] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "This patent describes an agentic browsing system that prevents prompt injection and protects user privacy through multiple security mechanisms. Capability tokens are time-boxed credentials issued by a capability broker, binding permissions to specific domains, path prefixes, action types, and preconditions. The system enforces strict separation between trusted content (system instructions and user goals) and untrusted content (page DOM, markup, scripts, metadata) through a context assembler schema. Content verification occurs through an injection detector that identifies prompt-injection indicators in HTML, markdown, CSS, image alt text, and microdata, assigning risk scores to determine transformation modes (redaction, summarization, removal). Tool calls are validated against the capability broker before execution, with least-privilege permissions scoped to specific sites, intents, and time windows. A human-in-the-loop interlock gates operations accessing credentials, initiating transactions, or exfiltrating data. The system operates within a secured computing environment with a local LLM, encrypted device-local vector store partitioned by domain, and excludes sensitive domain embeddings from remote inference."

[8.6] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "This patent describes a security-focused agentic browser framework implementing multiple protective mechanisms for sensitive transactions. Capability tokens are timeboxed and operation-specific, minted by a capability broker only after step-up authentication (passkey, biometric, or OTP). Decision envelopes consist of action summaries presented to users for co-approval, detailing destination, capabilities required, items/amounts, and risk indicators. Content verification uses injection detectors to identify malicious scripts, obfuscated instructions, and misleading markup in DOM, CSS, and image alt-text, redacting suspicious tokens before model processing. The system enforces strict separation through sanitized context for untrusted content while maintaining write-only autofill paths for credentials. Additional controls include spend/scope limits, rate limiting, capability revocation on navigation change or expiry, cross-origin blocking during sensitive operations, and immutable audit logs. The framework demonstrates comprehensive security architecture for agentic commerce by combining multi-factor authentication, granular capability scoping, content sanitization, and user-in-the-loop approval workflows."

[8.7] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "The patent describes a privacy-preserving agentic browser architecture that uses time-bounded capability tokens scoped to specific sites and intents to gov-

ern agent actions. Capability governance involves issuing tokens (S402), having a local LLM propose tool calls based on tokens (S404), validating proposed calls against tokens (S406), and permitting execution only when tokens match the designated site, intent, and active time window. A capability broker enforces user-configured limits on monetary exposure and operational scope, evaluating calls against per-domain and per-merchant budgets, transaction ceilings, rate limits, and category constraints. The system maintains strict separation between trusted and untrusted content: sensitive data (bank information, personal IDs, passwords) remains inaccessible to untrusted agents through protected interfaces and write-only paths. All analysis and operations are confined to a secured local environment with fail-closed rules blocking outbound transmission of raw credentials, financial data, and identifiers. Only anonymized summaries are transmitted externally, while embeddings and contextual data remain local in an encrypted, device-resident vector store."

[8.8] Security and Privacy Preserving Agentic Browser. Bao Tran. Patent (US) (2026). <https://patents.google.com/patent/US20260067335A1/en> Context: "This patent describes security mechanisms for AI browser agents conducting autonomous shopping transactions. Capability tokens enforce access controls by restricting credential use to authorized domains (e.g., Amazon.com), blocking requests to untrusted domains, and automatically revoking permissions on navigation changes or timeouts. The system uses write-only autofill paths to prevent payment credentials from being exposed to the model context. Sensitive financial and personal data are masked or redacted before being sent to external systems. An immutable audit log records all transactions with risk scores and outcomes. The system provides action traces showing proposed, executed, and blocked operations for user review. Session-based restrictions, step-up authentication, and user approval gates protect against unauthorized transactions. All telemetry outputs are anonymized, and embeddings from financial/retail domains are excluded from remote inference to maintain privacy and prevent adversarial content from manipulating the agent into unintended operations."

[9.1] SoK: Security of Autonomous LLM Agents in Agentic Commerce. Qian'ang Mao, Jiaxin Wang, Ya Liu, Li Zhu, Cong Ma, Jiaqi Yan. ArXiv (2026). <https://doi.org/10.48550/arxiv.2604.15367> Context: "This SoK paper systematically analyzes security challenges in autonomous LLM agents operating in financial/agentic commerce. The core finding is that autonomous financial-agent security is fundamentally cross-layer: threats originating in reasoning, tools, identity, or inter-agent interactions manifest as harm in custody, settlement, markets, or compliance. The paper proposes a five-layer defense architecture: (1) Prompt and Tool Hygiene—sanitizing external inputs, tagging agent-originated content, and verifying tool provenance before invocation to prevent public data from steering actions; (2) Verified Execution Context—using output validation, runtime context binding, and capability graphs to ensure plans match workflow and permitted sequences; (3) Payment Authorization and Custody; (4) Inter-Agent Trust Controls; and (5) Market and Compliance Monitoring. The paper emphasizes that point defenses are insufficient and that secure deployment requires coordinated controls across agent integrity, authorization, trust structures, market mechanisms, and regulatory compliance. It provides a five-dimensional taxonomy for analyzing autonomous financial-agent security."

[9.2] SoK: Security of Autonomous LLM Agents in Agentic Commerce. Qian'ang Mao, Jiaxin Wang, Ya Liu, Li Zhu, Cong Ma, Jiaqi Yan. ArXiv (2026). <https://doi.org/10.48550/arxiv.2604.15367> Context: "The SoK paper addresses security of autonomous LLM agents in agentic commerce, covering three main dimensions: prompt injection attacks (including direct injection via data feeds and memory injection), transaction authorization (credential management, spending policies, and intent verification), and inter-agent trust. The paper identifies security challenges specific to financial agents, such as how compromised tools can feed false information and how agents are vulnerable to prompt injection attacks. For content verification and separation, the paper proposes cryptographic provenance tags recording source, timestamp, and context hash for memory writes, with memory verifiers rejecting writes from untrusted sources. It emphasizes that agent permissions should be attenuated and contextual rather than broad, and highlights the challenge of verifying that an agent's reasoning intent matches its actual encoded transactions. The paper also discusses defense mechanisms against memory injection attacks and the importance of bounding agent security to the weakest tool integration."

[9.3] SoK: Security of Autonomous LLM Agents in Agentic Commerce. Qian'ang Mao, Jiabin Wang, Ya Liu, Li Zhu, Cong Ma, Jiaqi Yan. ArXiv (2026). <https://doi.org/10.48550/arxiv.2604.15367> Context: "This SoK paper by Mao et al. addresses security of autonomous LLM agents in agentic commerce. It defines autonomous financial agents as software systems powered by LLMs that maintain persistent financial state, execute transactions independently, and interact with external systems like blockchains and payment networks without per-transaction human approval. The paper identifies a critical security gap: existing frameworks don't address the full stack of concerns, as autonomous agents are simultaneously vulnerable as LLMs (prompt injection), blockchain actors (settlement risks), financial intermediaries (regulatory requirements), and multi-agent ecosystem participants (strategic manipulation). The paper makes five main contributions: (1) a unified threat taxonomy across five dimensions (agent integrity, transaction authorization, inter-agent trust, market manipulation, regulatory compliance), (2) cross-layer analysis showing how vulnerabilities at one layer propagate to others, (3) protocol security assessment of emerging agent-payment protocols, (4) a layered defense framework spanning prompt hardening, payment authorization, tool provenance, decentralized identity, and market safeguards, and (5) corpus-grounded synthesis analyzing academic papers, protocols, and incident evidence. The excerpt does not explicitly discuss trusted/untrusted content separation or content verification patterns."

[9.4] SoK: Security of Autonomous LLM Agents in Agentic Commerce. Qian'ang Mao, Jiabin Wang, Ya Liu, Li Zhu, Cong Ma, Jiaqi Yan. ArXiv (2026). <https://doi.org/10.48550/arxiv.2604.15367> Context: "This SoK paper addresses security of autonomous LLM agents in agentic commerce. It focuses on systems like OpenClaw that operate without continuous human oversight, managing digital assets and executing financial transactions. The paper organizes threats along five dimensions: agent integrity, transaction authorization, inter-agent trust, market manipulation, and regulatory compliance. It identifies 12 cross-layer attack vectors showing how vulnerabilities propagate from reasoning and tooling layers through custody and settlement to market harm and compliance issues. The authors propose a layered defense architecture to address authorization gaps in current agent-payment protocols. The paper emphasizes that securing agentic commerce requires coordinated cross-layer controls spanning LLM safety, protocol design, identity systems, market structure, and regulatory frameworks. However, the excerpt does not specifically discuss trusted/untrusted content separation or detailed reference architectures for agentic commerce security."

[9.5] SoK: Security of Autonomous LLM Agents in Agentic Commerce. Qian'ang Mao, Jiabin Wang, Ya Liu, Li Zhu, Cong Ma, Jiaqi Yan. ArXiv (2026). <https://doi.org/10.48550/arxiv.2604.15367> Context: "This SoK paper examines security of autonomous LLM agents in agentic commerce, focusing on the intersection of autonomous payment systems with open-ended LLM reasoning and untrusted tool use. It systematizes knowledge across five research communities including LLM security, autonomous agents, blockchain/DeFi security, financial technology, and multi-agent systems. The paper analyzes key protocols including the Agent Commerce Protocol (ACP) - a four-phase coordination layer for agent-to-agent commerce involving negotiation, transaction, evaluation, and settlement; the Model Context Protocol (MCP) which standardizes how LLM agents interact with external tools and services; Machine Payments Protocol (MPP); and x402. The analysis covers both deployed implementations and proposed features, with particular focus on blockchain-based and API-based agentic commerce representing the current frontier of autonomous agent deployment with real-asset exposure. The paper emphasizes novel security challenges arising from combining autonomous payment capabilities with open-ended LLM reasoning and untrusted tool invocation."

[9.6] SoK: Security of Autonomous LLM Agents in Agentic Commerce. Qian'ang Mao, Jiabin Wang, Ya Liu, Li Zhu, Cong Ma, Jiaqi Yan. ArXiv (2026). <https://doi.org/10.48550/arxiv.2604.15367> Context: "This SoK paper by Mao et al. (2026) examines security of autonomous LLM agents in agentic commerce. It defines four levels of agent autonomy (Advisory, Supervised, Delegated, and Fully Autonomous) and focuses primarily on security challenges of higher autonomy levels (2-3). The paper systematizes threats across five security dimensions: agent integrity, transaction authorization, inter-agent trust, market manipulation, and regulatory compliance. It derives 12 cross-layer attack vectors by analyzing how compromise at one system layer (reasoning, tools, custody, inter-agent protocols, settlement, oracles, identity, compliance) can induce harm at another layer while bypassing defenses. The analysis covers financial irreversibility, custody, settlement integrity, evaluator-mediated

commerce, and market manipulation risks in deployed or emerging agent-payment protocols. The excerpt does not specifically address trusted/untrusted content separation, content verification patterns, or detailed reference architectures for agentic commerce security."